



Федеральное государственное автономное образовательное

учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное
обеспечение

Дисциплина «Вычислительная математика»

Отчёт по лабораторной работе №6

Вариант - 6

Выполнил

Линейский Аким Евгеньевич

P3215

Проверил

Наумова Надежда Александровна

Санкт - Петербург 2026

Содержание

1. Содержание.....	2
2. Цель	3
3. Описание алгоритма решения задачи	3
4. Рабочие формулы.....	4
Задача Коши.....	4
Метод Эйлера	4
Метод Рунге-Кутты 4-го порядка.....	4
Метод Адамса	4
Правило Рунге.....	4
Аналитическая оценка	4
5. Программная часть	5
Блок схемы численных методов	5
Выполнение программы на различных исходных данных.....	6
Графики точного и полученного приближенного решений	6
Листинг программы.....	7
6. Вывод	9

Цель

Решить задачу Коши для обыкновенных дифференциальных уравнений численными методами.

Описание алгоритма решения задачи

1. Выбор дифференциального уравнения, ввод начальных условий x_0, y_0 .
Построение дискретной сетки узлов
2. Вычисление разгонного участка для многошаговых методов методом Рунге-Кутты 4-го порядка
3. Итерационный расчет по всей сетке
 - a. Метод Эйлера: пошагово вычисляются значения y_{i+1} на основе данных текущего узла (x_i, y_i)
 - b. Метод Рунге-Кутты 4-го порядка: для каждого шага рассчитываются четыре промежуточных коэффициента наклона (k_1, k_2, k_3, k_4) , определяющих средневзвешенное направление движения к точке y_{i+1}
 - c. Метод Адамса: начиная с индекса $i = 3$, алгоритм использует накопленные в памяти значения производных в точках $i, i-1, i-2, i-3$ для экстраполяции решения в узел $i+1$.
4. Оценка погрешностей вычислений
 - a. Для одношаговых методов (Эйлер, РК4) в каждом узле вычисляется локальная оценка по правилу Рунге путем сравнения ординаты, полученной с шагом h , и ординаты, полученной в результате двух половинных шагов по $h/2$.
 - b. Для многошагового метода Адамса оценка точности проводится глобально по всему отрезку интегрирования путем нахождения максимального абсолютного отклонения от точного решения в узлах сетки.

Рабочие формулы

Задача Коши

$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

Метод Эйлера

$$y_{i+1} = y_i + h * f(x_i, y_i)$$

Метод Рунге-Кутты 4-го порядка

$$y_{i+1} = y_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = f(x_i, y_i)$$

$$k_2 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right)$$

$$k_3 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_2\right)$$

$$k_4 = f(x_i + h, y_i + hk_3)$$

Метод Адамса

$$y_{i+1} = y_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$$

Правило Рунге

$$R = \frac{|y_h - y_{\frac{h}{2}}|}{2^p - 1}$$

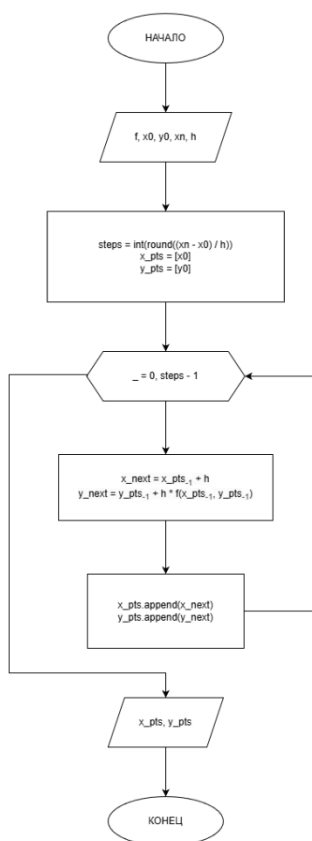
Аналитическая оценка

$$e = \max |y_{i \text{ точн}} - y_i|, \quad 0 \leq i \leq n$$

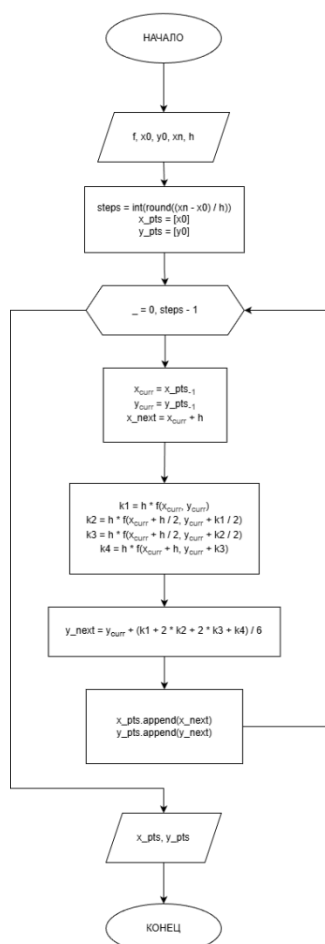
Программная часть

Блок схемы численных методов

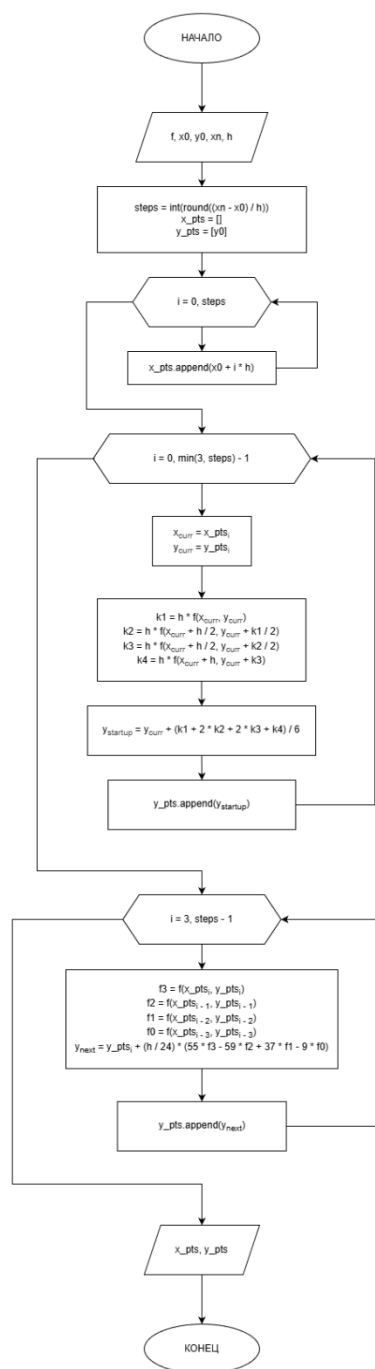
Метод Эйлера



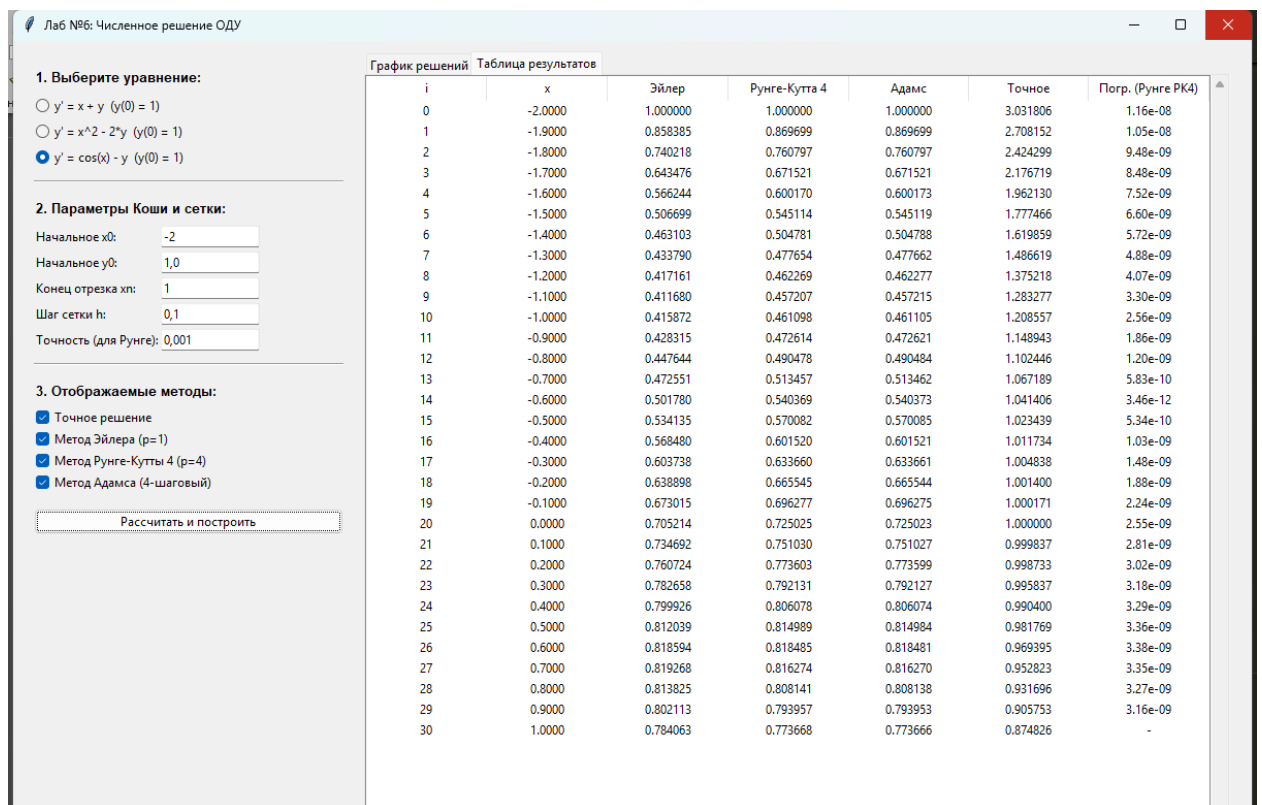
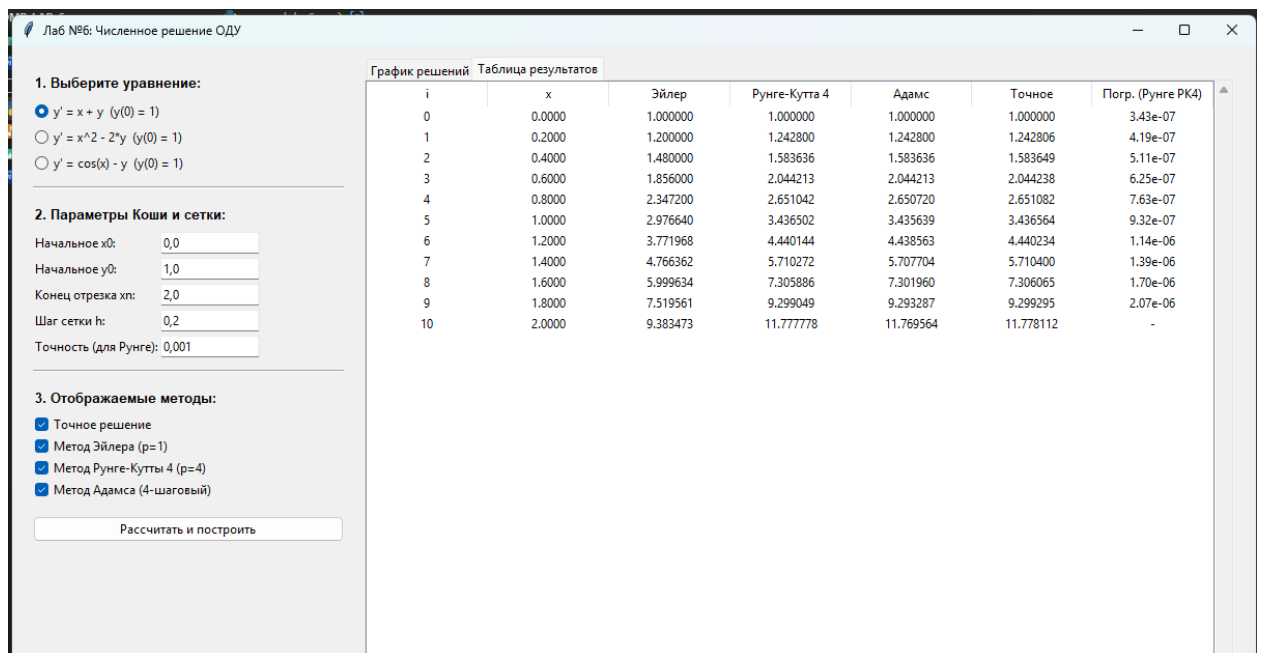
Метод Рунге-Кутты 4-го порядка



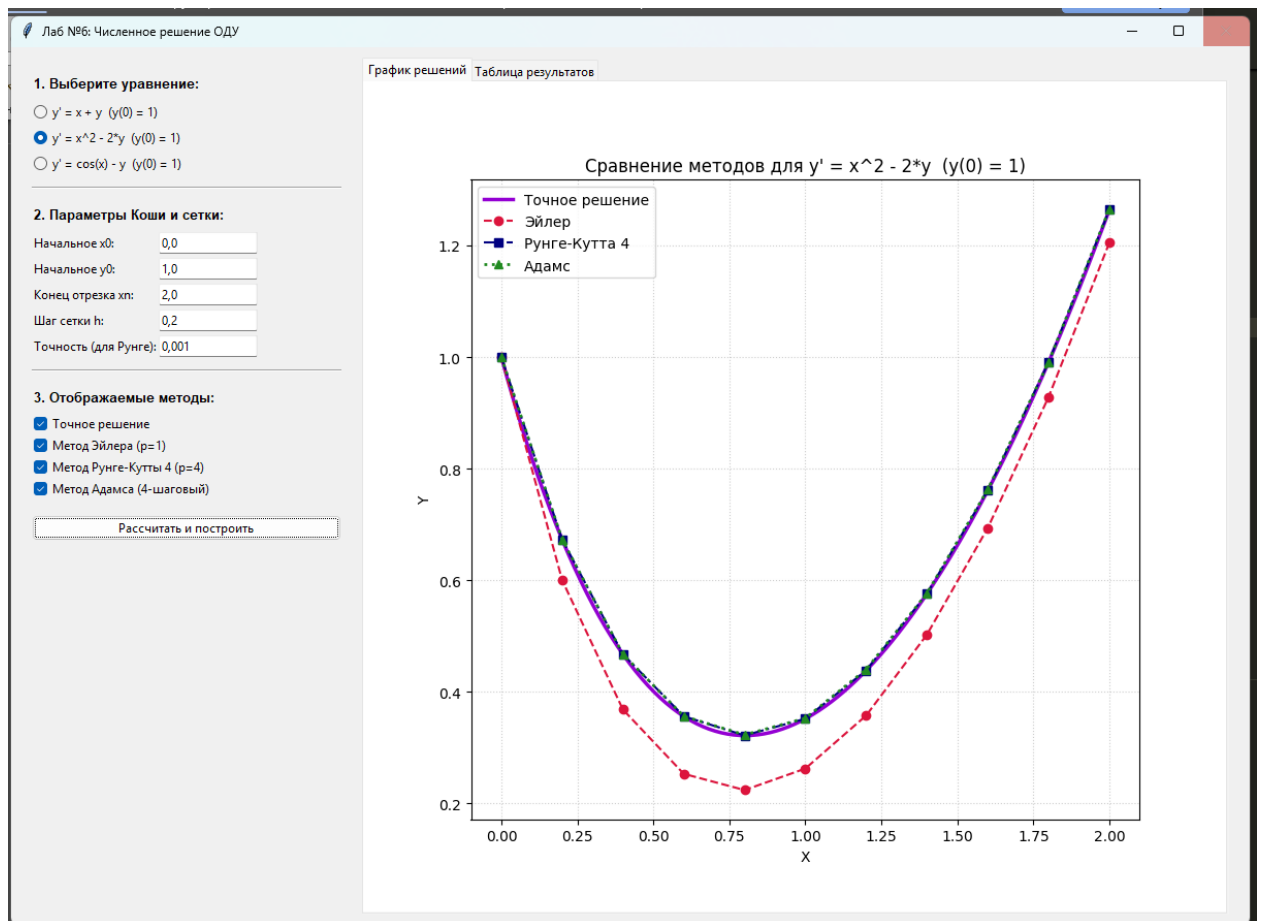
Метод Адамса



Выполнение программы на различных исходных данных



Графики точного и полученного приближенного решений



Листинг программы

```
class ODEMath:
    @staticmethod
    def get_equations():
        return [
            {
                "id": 1,
                "text": "y' = x + y (y(0) = 1)",
                "f": lambda x, y: x + y,
                "exact": lambda x, x0, y0: 2.0 * exp(x) - x - 1.0,
                "default_inputs": ("0,0", "1,0", "2,0", "0,2", "0,001"),
            },
            {
                "id": 2,
                "text": "y' = x^2 - 2*y (y(0) = 1)",
                "f": lambda x, y: x**2 - 2 * y,
                "exact": lambda x, x0, y0: 0.5 * x**2
                    - 0.5 * x
                    + 0.25
                    + 0.75 * exp(-2 * x),
                "default_inputs": ("0,0", "1,0", "2,0", "0,2", "0,001"),
            },
        ]
```

```

        {
            "id": 3,
            "text": "y' = cos(x) - y (y(0) = 1)",
            "f": lambda x, y: cos(x) - y,
            "exact": lambda x, x0, y0: 0.5 * sin(x) + 0.5 * cos(x) + 0.5 * exp(-x),
            "default_inputs": ("0,0", "1,0", "3,0", "0,2", "0,001"),
        },
    ]

    @staticmethod
    def euler_step(f, x, y, h):
        return y + h * f(x, y)

    @staticmethod
    def solve_euler(f, x0, y0, xn, h):
        steps = int(round((xn - x0) / h))
        x_pts = [x0]
        y_pts = [y0]
        for _ in range(steps):
            x_next = x_pts[-1] + h
            y_next = ODEMath.euler_step(f, x_pts[-1], y_pts[-1], h)
            x_pts.append(x_next)
            y_pts.append(y_next)
        return x_pts, y_pts

    @staticmethod
    def runge_kutta_4_step(f, x, y, h):
        k1 = h * f(x, y)
        k2 = h * f(x + h / 2, y + k1 / 2)
        k3 = h * f(x + h / 2, y + k2 / 2)
        k4 = h * f(x + h, y + k3)
        return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    @staticmethod
    def solve_rk4(f, x0, y0, xn, h):
        steps = int(round((xn - x0) / h))
        x_pts = [x0]
        y_pts = [y0]
        for _ in range(steps):
            x_next = x_pts[-1] + h
            y_next = ODEMath.runge_kutta_4_step(f, x_pts[-1], y_pts[-1], h)
            x_pts.append(x_next)
            y_pts.append(y_next)
        return x_pts, y_pts

    @staticmethod
    def solve_adams(f, x0, y0, xn, h):
        steps = int(round((xn - x0) / h))

        x_pts = []
        for i in range(steps + 1):
            x_pts.append(x0 + i * h)
        y_pts = [y0]

```



```

for i in range(min(3, steps)):
    y_pts.append(ODEMath.runge_kutta_4_step(f, x_pts[i], y_pts[i], h))

for i in range(3, steps):
    f3 = f(x_pts[i], y_pts[i])
    f2 = f(x_pts[i - 1], y_pts[i - 1])
    f1 = f(x_pts[i - 2], y_pts[i - 2])
    f0 = f(x_pts[i - 3], y_pts[i - 3])

    y_next = y_pts[i] + (h / 24) * (55 * f3 - 59 * f2 + 37 * f1 - 9 * f0)
    y_pts.append(y_next)

return x_pts, y_pts

@staticmethod
def estimate_runge_error(f, method_step_func, x, y, h, p):
    y_h = method_step_func(f, x, y, h)
    y_mid = method_step_func(f, x, y, h / 2)
    y_h2 = method_step_func(f, x + h / 2, y_mid, h / 2)
    return abs(y_h - y_h2) / (2**p - 1)

```

Код доступен по ссылке на удаленный репозиторий GitHub:

[Код программы](#)

Вывод

Выполнив данную лабораторную работу №6, я изучил различные способы решения ОДУ, реализовал численные методы и графический интерфейс с просмотром графиков найденных решений на языке Python.